

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Computer Vision with OpenCV

COURSE CODE: DSA02

COURSE OUTCOME COVERED

CO5: Apply computer vision techniques to real-world applications such as face recognition, tracking, medical imaging, and autonomous systems. (BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 2 Total Marks:30 Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I K. Pujitha (192524149) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

*I understand that any violation of academic integrity rules will result in disciplinary action. **Signature of the student:** K. Pujitha*

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	

Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. Scenario: Design an algorithm to detect faces in an image and recognize them using the Eigenfaces method.

Task:

- A. Input: A grayscale image (size $M \times N$)
- B. Output: Bounding boxes of detected faces and corresponding recognized names
- C. Requirements: Handle multiple faces in the same image and varying lighting conditions

Pseudocode Task:

1. Preprocess the image (grayscale normalization, histogram equalization)
2. Detect faces using a sliding window or Viola-Jones detector
3. Extract features using Eigenfaces
4. Compare with database of known faces using Euclidean distance
5. Output bounding boxes with recognized labels

2. Scenario: Design an algorithm to detect road lanes in a real-time driving video sequence.

Task:

- A. Input: Video frames from a front-facing camera
- B. Output: Lane lines highlighted on each frame
- C. Requirements: Robust to shadows, curves, and partial

occlusion Pseudocode Task:

1. Capture video frame
2. Convert frame to grayscale

3. Apply Gaussian smoothing to reduce noise
4. Perform edge detection (Canny)
5. Apply Hough Transform to detect straight/curved lines
6. Overlay detected lanes on the original frame
7. Repeat for all frames

1. Face Detection and Recognition Using Eigenfaces

Problem Understanding

- **Input:** Grayscale image of size $M \times N$
- **Output:** Bounding box ($x, y, width, height$) and recognized name for each face
- **Method:** Viola-Jones/sliding-window face detection + Eigenfaces for recognition
- **Special cases:** Multiple faces and varying illumination
- **Recognition:** Euclidean distance between projected face and database features

Pseudocode

ALGORITHM Eigenface_Face_Recognition

INPUT: Grayscale image I of size $M \times N$

OUTPUT: Bounding boxes and recognized names

1. $I \leftarrow \text{HistogramEqualization}(I)$
2. $I \leftarrow \text{NormalizeIntensity}(I)$
3. $\text{Faces} \leftarrow \text{ViolaJonesDetect}(I)$
4. IF Faces is empty THEN
 PRINT "No face detected"
 STOP
END IF
5. FOR each face F in Faces DO
6. $B \leftarrow \text{Crop}(I, F.\text{bounding_box})$
7. $B \leftarrow \text{Resize}(B, \text{Standard_Size})$
8. $B \leftarrow \text{HistogramEqualization}(B)$
9. $B \leftarrow \text{NormalizeIntensity}(B)$

```

10.      X ← Flatten(B)

11.      // Project face into Eigenface space
12.      Feature ← PCA_Project(X, Eigenfaces)

13.      MinDist ← INFINITY
14.      Name ← "Unknown"

15.      FOR each person P in FaceDatabase DO
16.          Dist ← EuclideanDistance(Feature, P.Feature)

17.          IF Dist < MinDist THEN
18.              MinDist ← Dist
19.              Name ← P.Name
20.          END IF
21.      END FOR

22.      IF MinDist > Threshold THEN
23.          Name ← "Unknown"
24.      END IF

25.      Output(F.bounding_box, Name)

26. END FOR

END ALGORITHM

```

Explanation of the Main Steps

1. Preprocessing

Histogram equalization improves contrast in poorly illuminated images. Intensity normalization reduces the effect of different lighting conditions.

2. Face Detection

`ViolaJonesDetect(I)` scans the image and returns the bounding boxes of all detected faces, so multiple people can be handled.

3. Feature Extraction

Each detected face is resized to the same size and converted into a vector. PCA projects this vector onto the previously trained Eigenface space.

4. Recognition

The extracted feature is compared with every known person's stored Eigenface feature using:

$$\text{Euclidean Distance} = \sqrt{\sum (\text{Feature} - \text{DatabaseFeature})^2}$$

The person with the minimum distance is selected.

5. Unknown Face Handling

If the minimum distance is greater than a predefined threshold, the face is labeled "**Unknown**" rather than incorrectly assigning a known person's name.

Flow

Input Image → Histogram Equalization → Normalization → Face Detection → Crop Faces → Resize → Eigenface/PCA Feature Extraction → Euclidean Distance → Threshold Check → Name + Bounding Box

Why this satisfies the rubric

- **Problem Understanding:** Clearly defines input, output, multiple faces, and lighting variation.
- **Algorithm Design:** Complete pipeline from preprocessing to recognition.
- **Control Structures:** Uses `FOR` loops and `IF-ELSE` conditions for multiple faces, database comparison, and unknown-face detection.
- **Pseudocode Structure:** Uses clear steps, functions, inputs, outputs, and termination conditions.
- **Completeness:** Handles no face, multiple faces, varying lighting, known faces, and unknown faces.

2. Real-Time Road Lane Detection

Problem Understanding

The objective is to design an algorithm that detects road lanes from a **real-time video captured using a front-facing camera**. Each video frame is processed to identify lane markings and highlight them on the original frame.

The algorithm should work effectively even when there are **shadows, curved roads, noise, and partial occlusion** of lane markings.

Input

- Real-time video sequence.
- Frames captured from a front-facing camera.

Output

- Original video frames with detected left and right lane lines highlighted.

Requirements

- Reduce image noise.
- Handle different lighting and shadows.
- Detect straight and curved lanes.
- Handle partially hidden lane markings.
- Process frames continuously in real time.

Algorithm

1. Capture a frame from the front-facing camera.
2. Resize the frame for faster processing.
3. Convert the frame into grayscale.
4. Apply Gaussian smoothing to remove noise.
5. Adjust brightness and contrast to reduce shadow effects.
6. Apply Canny edge detection.
7. Select the road region using Region of Interest (ROI).
8. Apply Hough Transform to detect lane candidates.
9. Separate the detected lines into left and right lanes.
10. Remove false lines using slope and position conditions.
11. Check whether lane markings are partially occluded.
12. Use information from the previous frame to estimate missing lane portions.
13. For curved roads, use detected points to estimate the lane curve.
14. Smooth the lane positions between consecutive frames.
15. Draw the detected lane lines on the original frame.
16. Highlight the road area between the lane lines.
17. Display the processed frame.
18. Repeat the process for every frame until the video ends.

Detailed Pseudocode

ALGORITHM RealTimeRoadLaneDetection

INPUT:

Video frames from a front-facing camera

OUTPUT:

Video frames with highlighted road lanes

BEGIN

1. Initialize video camera.
2. Initialize previous_left_lane
and previous_right_lane.
3. WHILE video is running DO
 4. Capture current frame.
 5. IF frame is not available THEN
BREAK
END IF
 6. Resize the frame.
 7. Convert the frame into grayscale.
 8. Apply Gaussian smoothing
to remove image noise.
 9. Adjust brightness and contrast
to reduce shadow effects.
 10. Apply Canny edge detection
to identify strong edges.
 11. Define Region of Interest (ROI)
around the road area.
 12. Remove edges outside the ROI.

```
13. Apply Hough Transform
    to detect possible lane lines.

14. FOR each detected line DO

    15. Calculate its slope
        and position.

    16. Remove lines having
        invalid slope or position.

END FOR

17. Divide valid lines into
    left-lane and right-lane candidates.

18. IF enough lane points are detected THEN

    19. Estimate left and right
        lane positions.

ELSE

    20. Use previous frame's
        lane positions.

END IF

21. IF lane marking is partially occluded THEN

    22. Estimate the missing portion
        using previous lane information.

END IF

23. IF road is curved THEN

    24. Use multiple detected points
        to estimate the lane curve.
```



```
        END IF

    25. Smooth the lane positions
        between consecutive frames.

    26. Update previous_left_lane
        and previous_right_lane.

    27. Draw the left lane on the frame.

    28. Draw the right lane on the frame.

    29. Highlight the region between
        the detected lanes.

    30. Display the processed frame.

    31. Move to the next video frame.

END WHILE

32. Stop the camera.

33. Release video resources.

END
```

Handling Special Conditions

1. Shadows:

Brightness/contrast adjustment and grayscale conversion reduce the effect of shadows. Gaussian smoothing also removes small unwanted variations.

2. Curved Roads:

Instead of depending only on one straight line, multiple edge points are used to estimate the shape of a curved lane.

3. Partial Occlusion:

If a vehicle or another object temporarily blocks a lane marking, the algorithm uses the **previous frame's lane information** to estimate the missing lane.

4. False Lane Detection:

Slope, position, and ROI conditions are used to remove edges that are unlikely to belong to road lanes.

5. Real-Time Processing:

A loop processes one frame at a time and immediately displays the detected lanes.

Control Structures Used

- **WHILE loop:** Processes continuous video frames.
- **FOR loop:** Examines each detected line.
- **IF-ELSE conditions:** Handle shadows, missing lanes, curves, and occlusion.
- **Functions/modules:** Grayscale conversion, Gaussian filtering, Canny detection, ROI selection and Hough Transform.